A Unified Framework for Fidelity-Aware Quantum Routing

Abstract—Quantum networks require routing algorithms that simultaneously enforce per-hop fidelity constraints and sustain high throughput across concurrent requests. Most existing fidelity-aware approaches (heuristic, integer linear programming, and single-agent reinforcement learning) process requests one at a time or fall back to serialised per-request scheduling under contention. When entanglement generation rates are low (quality-limited regime), sequential routing is optimal: the agent can give each request full global-state attention. As generation rates rise toward near-term hardware targets, however, sequential processing leaves the majority of available links idle and subject to decoherence between decision steps, creating a throughput ceiling that no amount of training or tuning can overcome. This is the capacity-limited regime, where the routing bottleneck is structural rather than algorithmic. This paper presents *QuRA*, a family of reinforcement-learning-based routing algorithms that matches the architecture to the regime. The sequential variant (QuRA-Seq) is optimized for the quality-limited regime, matching ILP routing quality while running 79% faster. For the capacity-limited regime, we introduce three parallel multi-agent variants: QuRA-Flock (independent per-request agents), QuRA-Guard (post-hoc conflict resolution), and QuRA-Hive (QMIX cooperative training with centralized training and decentralized execution). All variants enforce fidelity thresholds at every hop and generalize across topologies without retraining. We derive an optimal routing window $W^* = 1/\kappa$, where κ is the network's congestion coefficient, and validate it empirically. Experiments on grid and real-world Surfnet topologies show that QuRA-Hive achieves over $12\times$ throughput improvement over sequential routing and $4\times$ fewer overbooking conflicts than rule-based resolution, while maintaining strict fidelity compliance.

Index Terms—Quantum Network, Entanglement routing, Multi-agent reinforcement learning, Parallel routing, QMIX, Throughput optimization, Fidelity-aware routing

1 Introduction

The quantum internet [1], promising to revolutionize secure communication [2]–[5], distributed quantum computing (DQC) [6]–[8], and quantum-enhanced sensing [9], [10], relies on *quantum routing* to establish high-quality entanglement between distant nodes. Unlike classical routing, this task faces unique challenges: fidelity degradation from entanglement swapping (Fig. 1) [11], probabilistic link availability, and single-use entangled links that decohere if not consumed within a coherence window [12], [13]. These constraints necessitate scalable, fidelity-aware solutions for the dynamic, multi-request scheduling problem.

Several approaches have been proposed. Shortest-path methods such as qDijkstra [14] ignore fidelity loss. Q-CAST [15] handles concurrent requests through redundant path provisioning but does not account for fidelity decay from swapping. Integer linear programming (ILP) methods such as REPS [16] provide bounded sub-optimality guarantees via LP relaxation and rounding, but are not scalable to large networks. Reinforcement learning (RL) based approaches [17]–[19] learn adaptive policies but typically ignore fidelity constraints or handle only single-request scenarios.

The core challenge motivating this work is illustrated in Fig. 2: the shortest paths between two source-destination pairs yield end-to-end fidelities below the required threshold, while slightly longer paths chosen for their fidelity profile satisfy the requirement. Fidelity-aware routing is therefore not optional but essential. However, existing fidelity-aware methods, including our prior QuRA algorithm [20], process routing decisions for each request sequentially using a single agent. While this gives the agent full global-state attention for each decision, it creates a structural limitation under high concurrent load: entangled links available to waiting requests sit idle and decohere while the system advances one request at a time.

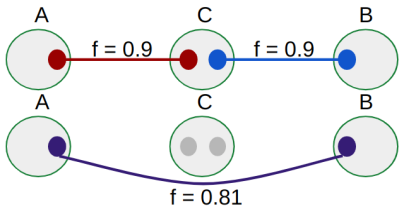


Fig. 1. Entanglement swapping: two independent entangled links are combined at an intermediate repeater to produce a long-distance entangled pair. Fidelity degrades at each swap according to $F_{\text{swap}} = F_1F_2 + \frac{(1-F_1)(1-F_2)}{3}$ [12].

The severity of this idle-link waste depends on the network's *operating regime*, set by the entanglement generation rate (EGR) relative to the coherence window. We therefore distinguish two regimes that motivate two complementary designs. In the *quality-limited regime*, EGR is slow (as in current NV-center [21] or trapped-ion hardware typically used for high-fidelity long-distance QKD links and small-scale distributed quantum computing [8]). The network produces few links per slot, idle-link waste is negligible, and the bottleneck is per-path fidelity optimisation; sequential routing with centralized attention is the right design. In the *capacity-limited regime*, EGR is fast (multiplexed photonic sources [22]–[24] and high-rate quantum backbones

envisioned for data-center DQC, quantum-enhanced sensor networks, and metropolitan-scale QKD). The link supply exceeds what a single agent can consume within the coherence window, idle-link decoherence dominates, and parallel execution becomes essential. A routing framework that ignores this dichotomy pays a permanent cost in at least one regime.

This paper presents *QuRA*, a family of reinforcement learning based routing algorithms spanning two complementary architectures matched to these regimes. The **sequential single-agent variant (QuRA-Seq)** uses a Deep Q-Reinforcement Learning (DQRL) agent that makes fine-grained, per-hop decisions across concurrent requests, enforcing fidelity thresholds and generalizing across topologies without retraining. It achieves routing quality comparable to ILP while reducing computation time by up to 79%, making it the appropriate design for the quality-limited regime.

To address the capacity-limited regime, we introduce a **Parallel Multi-Agent Deep Reinforcement Learning (MARL) routing framework** that assigns one dedicated RL agent per active request, enabling all agents to infer next-hop decisions simultaneously. We propose three variants: **QuRA-Flock** (basic parallel execution without coordination), **QuRA-Guard** (parallel with Q-value and shortest-path priority conflict resolution), and **QuRA-Hive** (QMIX [25] cooperative MARL with centralized training and decentralized execution, CTDE). All variants enforce fidelity thresholds at every hop; the variants differ in coordination mechanism and the resulting throughput–overhead trade-off.

Our key contributions are:

- A hardware-grounded two-regime framework that explains when sequential routing is optimal and when parallel execution becomes essential, characterises the crossover analytically (Section 4.4), and derives the optimal routing window $W^* = 1/\kappa$ (Section 5.5).
- A parallel MARL routing framework with three coordination variants (QuRA-Flock, QuRA-Guard, QuRA-Hive) that progressively resolve inter-agent link conflicts under per-hop fidelity constraints, from no coordination to post-hoc rule-based resolution to proactive QMIX cooperative training (Section 4.5).
- The first integration of QMIX into quantum routing: per-request agents are trained jointly via centralized training with decentralized execution, avoiding runtime inter-agent communication while still coordinating link claims (Section 4.5.4).
- Empirical validation on grid and real-world Surfnet topologies showing QuRA-Hive achieves over $12\times$ throughput improvement over sequential routing and $4\times$ fewer link-booking conflicts than rule-based resolution, while maintaining strict per-hop fidelity compliance (Section 5).

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the system model. Section 4 presents the QuRA routing framework. Section 5 presents evaluation results. Section 6 concludes.

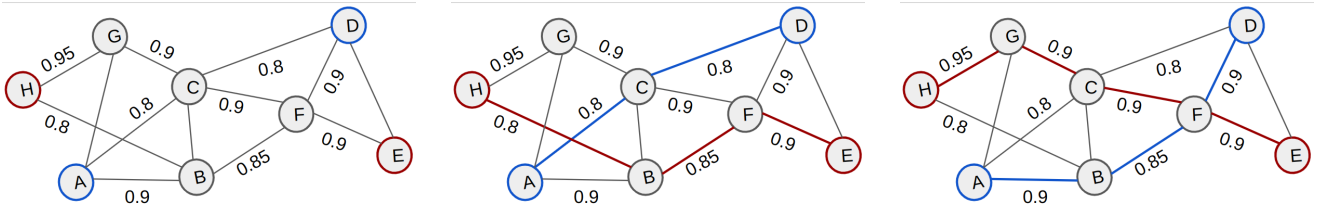
2 Related Work

Quantum routing has been studied through three broad lines of work. We refer the reader to [26] for a comprehensive survey and to [27]–[29] for broader treatments of quantum network protocol stacks and communication systems; here we focus on the works most relevant to our contribution.

Heuristic methods adapt classical graph algorithms to the quantum setting. qDijkstra [14] uses entanglement fidelity as edge weight but ignores link capacity. Caleffi [30] formulates optimal routing on a quantum network graph but considers only single source-destination pairs. Q-CAST [15] provisions redundant paths across multiple pairs but does not account for fidelity decay from swapping. Li et al. [31] enforce purification at each segment but all decisions remain static. Li et al. [32] provide fidelity-guaranteed entanglement routing with analytical bounds but rely on static path computation. Hu et al. [33] propose a high-fidelity routing scheme with local optimization but without scalable multi-request support. COSP [34] integrates Monte Carlo Tree Search with conflict resolution for concurrent requests, handling multi-path planning without learned policies. Zeng et al. [35] present a comprehensive entanglement routing design framework using graph-theoretic approaches but without learned adaptivity. Cicconetti et al. [36] study request scheduling in quantum networks from a queuing perspective, complementary to our routing focus.

ILP-based methods provide provable guarantees at the cost of scalability. REPS [16] maximizes throughput through redundant provisioning but scales exponentially. Chakraborty et al. [37] formulate entanglement distribution as a multicommodity flow problem, providing theoretical throughput bounds but not addressing fidelity decay. FENDI [38] characterizes the throughput-fidelity trade-off with a polynomial-time approximation, but handles only a single pair. Nguyen et al. [39] develop approximation algorithms for maximizing entanglement routing rate with provable guarantees but do not incorporate decoherence dynamics. PSC [40] jointly optimizes purification scheduling across concurrent requests but operates on fixed paths with no learning. Yang et al. [41] address asynchronous entanglement provisioning for distributed quantum computing workloads but assume pre-computed paths.

Reinforcement learning overcomes scalability limitations while enabling policies to improve through interaction [42]. DQRA [17] was among the first to apply deep RL to multi-request quantum routing but omits fidelity degradation. Roik et al. [19] explore tabular RL for small quantum networks, demonstrating the potential of learned routing but without scaling to realistic network sizes. Sharma et al. [18] apply deep RL to routing and resource assignment in QKD-secured optical networks, a related but distinct problem domain. RELiQ [43] adopts a decentralized multi-agent architecture in which each network node runs an independent Graph Neural Network (GNN) agent. This design achieves strong scalability to large topologies; each node optimizes its local forwarding decisions independently, without a mechanism for coordinating link allocations across simultaneous requests. QuDQN [44] addresses a complementary gap by integrating fidelity degradation, qubit memory capacity, and



(a) Sample network with two requests (A,D) and (E,H), each requiring minimum fidelity of 0.7.

(b) Hop-based shortest paths (A–C–D and E–F–B–H) yield fidelities of 0.653 and 0.63, failing the threshold.

(c) Fidelity-aware paths (A–B–F–D and E–F–C–G–H) achieve fidelities of 0.70 and 0.705, satisfying the threshold for both requests.

Fig. 2. Motivation for fidelity-aware routing. Hop-based shortest paths (Fig. 2(b)) fail to meet the fidelity threshold due to fidelity degradation from entanglement swapping, while longer paths selected for their fidelity profile (Fig. 2(c)) satisfy the requirement for both requests.

probabilistic swapping rates into a single unified DQN formulation, representing the most complete single-agent treatment of the fidelity-resource trade-off to date. However, QuDQN routes requests one at a time using a centralized controller, inheriting the same sequential idle-link bottleneck as prior single-agent methods and not scaling to high-concurrency regimes. QuRA-Hive differs from both: it assigns per-request agents that act simultaneously (unlike RELiQ's per-node decomposition), enforces fidelity at every hop decision rather than relying on post-hoc path filtering, and coordinates agents jointly through QMIX training to avoid link conflicts proactively (unlike QuDQN's single-agent formulation). DyQNet [45] optimizes dynamic entanglement routing with online request handling but focuses on throughput rather than per-hop fidelity enforcement. Rehman et al. [46] consider flexible routing in heterogeneous quantum networks but without multi-agent coordination.

Multi-agent RL in networking. Cooperative MARL has been applied to a range of classical network resource-allocation problems, including spectrum management, traffic signal control, and packet routing [47], where centralized training with decentralized execution (CTDE) balances coordination quality against runtime scalability. QMIX (Rashid et al. [25]) is a widely-used value-decomposition method in this space, and we adopt it here as a proven fit for scenarios in which agents must coordinate shared resources during training but must act independently on local observations at inference. In quantum routing the same tension arises: training can leverage global topology and all agents' positions, while inference must be fast enough to act before links decohere. Our work introduces the first cooperative MARL framework for quantum routing, where per-request agents are trained jointly to maximise network-wide fidelity-compliant throughput.

Reviewing these works reveals a consistent gap. Five properties are jointly required for high-performance routing under concurrent load: (i) fidelity awareness [12], (ii) multi-request support [15], (iii) learned adaptive policies [17], (iv) throughput orientation [16], and (v) parallel execution. No prior work achieves all five. Heuristic methods handle concurrent requests but ignore fidelity decay from swapping. ILP methods enforce fidelity but scale exponentially. Single-agent RL methods (DQRA, QuDQN, DyQNet) learn fidelity-aware policies but route requests one at a time, inheriting the idle-link decoherence bottleneck that persists regardless of policy quality. Multi-agent methods (RELiQ) achieve parallel per-node routing but decompose the problem node-by-node rather than request-by-request, and do not coordinate link allocations across simultaneous requests.

The root cause is architectural: routing one request at a time is correct only when link supply is tight enough that waiting requests have no links to lose. As entanglement generation rates increase toward near-term hardware targets, this assumption breaks down, and parallelism becomes essential. QuRA is the first work to make this regime dependence explicit, formalize the crossover condition, and provide matched algorithms for both regimes. QuRA-Hive is the first method to satisfy all five properties simultaneously through cooperative MARL with per-hop fidelity enforcement.

3 System Model

3.1 Quantum Network Basics

A quantum network enables the distribution of entangled quantum states between distant nodes through a chain of quantum repeaters [11], [48]. The fundamental resource is an *entangled link*: a Bell pair shared between two adjacent nodes, generated by emitting entangled photons and performing a Bell-state measurement [49]. Entangled links are probabilistic (generation succeeds with some probability that depends on the optical channel and hardware quality) and *single-use*: once a link is consumed by a swapping operation it is destroyed and must be regenerated. Each node is equipped with *quantum memory* to store generated links while waiting for the rest of the path to be established [50]. Memory is limited in both capacity (number of storable qubits) and lifetime (coherence time), after which stored entanglement is irreversibly lost. Long-distance entanglement is established through *entanglement swapping* at intermediate repeater nodes: two adjacent links are jointly measured, consuming both and producing a new entangled pair between the outer endpoints. Fidelity, a value in $[0, 1]$ quantifying how close the state is to a perfect Bell pair, degrades at each swap and through memory decoherence, making fidelity management a central challenge in quantum routing [1].

3.2 Network Model

We model a quantum network as an undirected graph $G = (V, E)$ (Fig. 3), where V is the set of quantum repeater nodes and E is the set of physical communication links.

Each node $v \in V$ is equipped with quantum memory of capacity M_v qubits. Each physical link $(u, v) \in E$ supports $c_{uv} \geq 1$ parallel quantum channels, allowing multiple entangled pairs to be generated simultaneously on the same edge. Each channel has an entanglement generation probability $p_{uv} \in (0, 1]$ and an initial link fidelity that depends on the physical distance d_{uv} between nodes:

$$F_{uv}^0 = e^{-\alpha\, d_{uv}}, \tag{1}$$

where $\alpha > 0$ is a channel attenuation coefficient capturing photon loss and noise over the fiber [51], [52]. Longer links produce lower-fidelity Bell pairs, making path selection sensitive to both hop count and distance.

3.3 Time Slot Model and Entanglement Dynamics

Network operation proceeds in discrete time slots indexed by $t \in \{1, 2, \ldots\}$. Each time slot corresponds to an absolute duration Δt (seconds). At the start of each time slot, entanglement generation is attempted on every channel of every link independently: a Bell pair is created with probability p_{uv} and stored in the quantum memories of nodes u and v.

Stored entangled pairs decohere over time. A pair stored for an absolute duration Δt_{store} after creation degrades in fidelity as:

$$F(\Delta t_{\text{store}}) = F_0 \cdot e^{-\Delta t_{\text{store}}/T_c}, \tag{2}$$

where F_0 is the initial fidelity at generation and T_c is the memory coherence time [53], [54]. At the end of each time slot, any entangled pair not consumed by a swap or delivery is discarded, and entanglement generation is reattempted afresh in the next slot. This perishability is the physical motivation for parallel routing: links that sit idle while the system attends to other requests are permanently lost.

3.4 Fidelity Model and Entanglement Swapping

End-to-end entanglement along a path $P = (v_0, v_1, \ldots, v_k)$ is established by performing swapping operations at each intermediate repeater $v_1, \ldots, v_{k-1}$. Each operation combines two adjacent entangled pairs, consuming both and producing a new pair between the outer endpoints.

In practice, Bell pairs generated over lossy channels are well approximated as Werner states [12], a mixture of a perfect Bell state and maximally mixed noise. This approximation is well-supported by experiment: entanglement sources based on nitrogen-vacancy centers in diamond [54], trapped-ion systems [50], and photonic interfaces [52] all produce states whose off-diagonal structure is dominated by isotropic depolarizing noise, consistent with the Werner form. The approximation is valid as long as depolarizing noise dominates over coherent or asymmetric errors, which holds for the fiber-based repeater architectures we consider. For two Werner-state links with fidelities F_1 and F_2, the fidelity after swapping is:

$$F_{\text{swap}}(F_1, F_2) = F_1 F_2 + \frac{(1 - F_1)(1 - F_2)}{3}. \tag{3}$$

This formula applies because the Werner state approximation is preserved under Bell-state measurements and photon loss, even when the initial generation targets a Bell state [12]. Since $F_{\text{swap}}(F_1, F_2) < \min(F_1, F_2)$ for all $F_1, F_2 < 1$, fidelity degrades strictly with each additional hop, motivating fidelity-aware path selection as illustrated in Fig. 2.

3.5 Request Model

End-to-end entanglement delivery follows four steps, consistent with the model in Q-CAST [15] and the link-layer protocol of Dahlberg et al. [55]: (1) *Entanglement generation*: each link independently attempts to generate Bell pairs using an Adaptive Entanglement Generation (AEG) protocol [56]; successful pairs are stored in quantum memory. (2) *Path selection*: given the current set of available links, a routing algorithm selects the next-hop link for each active request. (3) *Entanglement swapping*: intermediate repeaters perform Bell-state measurements to extend entanglement along the selected path. (4) *Delivery*: the end-to-end pair is consumed by the application (QKD, DQC, sensing) if its fidelity meets $F^{\min}$.

Scope of this work. We focus on Step 2 (path selection), which is the decision-making bottleneck, as illustrated in Fig. 4. Entanglement generation (Step 1) is handled by the AEG protocol, which generates links asynchronously and independently on each edge. Swapping (Step 3) and delivery (Step 4) follow deterministically from the selected path. The RL agent's action at each time step is to choose the next-hop link for each active request, and the fidelity constraint is enforced at every hop decision.

A routing request $r_i = (s_i, d_i, F_i^{\min})$ specifies a source node $s_i \in V$, a destination node $d_i \in V$, and a per-request minimum fidelity threshold $F_i^{\min} \in (0.5, 1]$. We adopt a uniform threshold $F^{\min}$ across all requests in our evaluation, consistent with prior work [20], as the routing algorithm operates without knowledge of individual thresholds at decision time.

A batch of N concurrent requests $\mathcal{R} = \{r_1, \ldots, r_N\}$ arrives at the start of a *routing window* of W consecutive time slots. Within this window, each request r_i is retried across multiple time slots: if a routing attempt fails in slot t due to unavailable links or fidelity violation, the request is reattempted in slot $t + 1$. A request is discarded after K failed consecutive attempts or when the routing window expires, whichever comes first. A request is considered *successfully served* if an end-to-end entangled pair is delivered with fidelity at least $F^{\min}$ within the window. The routing window parameter W controls the trade-off between allowing sufficient time for multi-hop paths to complete and minimizing idle-link decoherence waste; its optimal value is characterized analytically in Section 5.5.

3.6 Performance Objectives

We define two complementary metrics, both fidelity-constrained, corresponding to two operating regimes described in the introduction.

Success Rate (SR) measures the fraction of requests served, consistent with the metric used in prior quantum routing evaluations [15], [17]:

$$\text{SR} = \frac{|\{r_i \in \mathcal{R} : r_i \text{ served}\}|}{|\mathcal{R}|}. \tag{4}$$

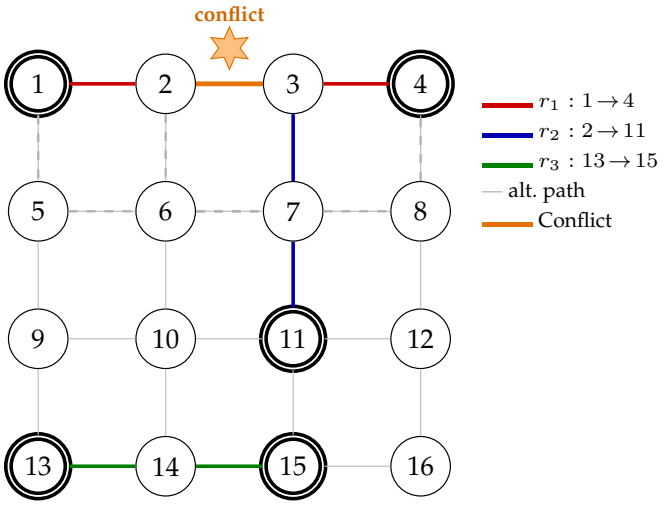


Fig. 3. Quantum network model: a 4×4 grid with three concurrent routing requests. Solid lines show chosen paths; dotted lines show alternate paths. Requests r_1 (1→4) and r_2 (2→11) contest link 2→3 (orange label). Both also have alternate paths through node 6 (orange circle), creating a second potential conflict. Intelligent path selection must resolve these conflicts while maintaining end-to-end fidelity above $F^{\min}$. Double-bordered nodes are source/destination endpoints.

SR is the dominant metric in the quality-limited regime, where the challenge is finding high-fidelity paths.

Throughput (TP) measures fidelity-compliant deliveries per time slot, extending the throughput notion of REPS [16] and Lee et al. [57] with an explicit fidelity gate:

$$\text{TP} = \frac{|\{r_i \in \mathcal{R} : r_i \text{ served}, \; F_i \geq F^{\min}\}|}{W}. \tag{5}$$

TP is the dominant metric in the capacity-limited regime, where concurrent requests exceed what sequential processing can serve before decoherence. Deliveries failing to meet $F^{\min}$ are not counted.

3.7 Problem Statement

As illustrated in Fig. 2, shortest-path routing fails to meet fidelity thresholds because each entanglement swap degrades quality. The problem is to select next-hop links at each step for N concurrent requests over a routing window of W time slots, such that the delivered end-to-end fidelity meets $F^{\min}$ for each request. Links are probabilistic, single-use, and decohere in memory, making this a sequential decision problem under uncertainty.

The appropriate objective depends on the operating regime: SR (Eq. (4)) when load is low and fidelity precision matters most, TP (Eq. (5)) when load is high and idle-link decoherence dominates. Both enforce $F^{\min}$ at every hop. This motivates two complementary architectures: a sequential agent for the quality-limited regime and a parallel multi-agent framework for the capacity-limited regime.

4 QuRA Routing Framework

4.1 Framework Overview

QuRA is a family of reinforcement learning based routing algorithms designed around two complementary architectures that target distinct operating regimes of a quantum network, as summarised in Fig. 5.

The design starts from a physical observation. When entanglement generation is slow, links are scarce and their fidelity is the dominant bottleneck. Processing requests one at a time gives the agent full global-state attention and maximises per-path fidelity; this is QuRA-Seq (Section 4.3). As entanglement generation rates increase through hardware advances, however, the network begins to produce more links per coherence window than one agent can consume. Serialising requests then *wastes* capacity: links for waiting requests decohere while the agent attends to an earlier one. This structural bottleneck motivates the transition to parallel execution (Section 4.4).

Parallelism introduces a new coordination challenge. Deploying N independent agents eliminates the idle-link waste but creates link-booking conflicts: multiple agents may simultaneously select the same scarce link. QuRA addresses this with a three-step coordination ladder (Section 4.5). QuRA-Flock provides the parallel baseline: N agents act independently with no conflict awareness. QuRA-Guard adds a post-hoc conflict resolution step that detects and breaks booking conflicts after agents have acted, limiting the damage but not preventing it. QuRA-Hive goes further: using QMIX cooperative training, agents learn to anticipate the actions of their peers and route around congestion proactively, reducing conflicts before they arise rather than resolving them after.

All four variants share the same five-stage decision pipeline (Fig. 5), the same observation space, reward structure, and action-masking mechanism, and the shared routing principles described in Section 4.2. Only the **Decide & Resolve** stage (and the coordination mechanism within it) differs across variants. This controlled design allows a clean ablation of each coordination mechanism in Section 5.

This section is organised as follows: shared routing principles (Section 4.2), the sequential variant (Section 4.3), the regime-transition bottleneck analysis (Section 4.4), the three parallel variants (Section 4.5), and a computational complexity comparison (Section 4.6). The optimal routing window analysis is presented alongside empirical validation in Section 5.5.

4.2 Shared Routing Principles

All QuRA variants, sequential and parallel alike, share four core routing principles described once here and inherited by every variant.

Hop-by-hop decision making. Rather than computing full source-to-destination paths, each variant builds paths incrementally: at each decision step, the routing agent selects the next hop for one request (sequential) or all active requests (parallel). This avoids the combinatorial explosion of path enumeration while enabling real-time adaptation to link failures and fidelity changes.

Fidelity enforcement at every hop. Before executing any next-hop decision, the agent verifies that the projected end-to-end fidelity, computed recursively via Eq. (3), remains above $F^{\min}$. If the fidelity check fails, the request is marked as failed immediately. This enforcement is identical across all variants and ensures that *every* delivered path meets the fidelity threshold, regardless of whether routing is sequential or parallel.

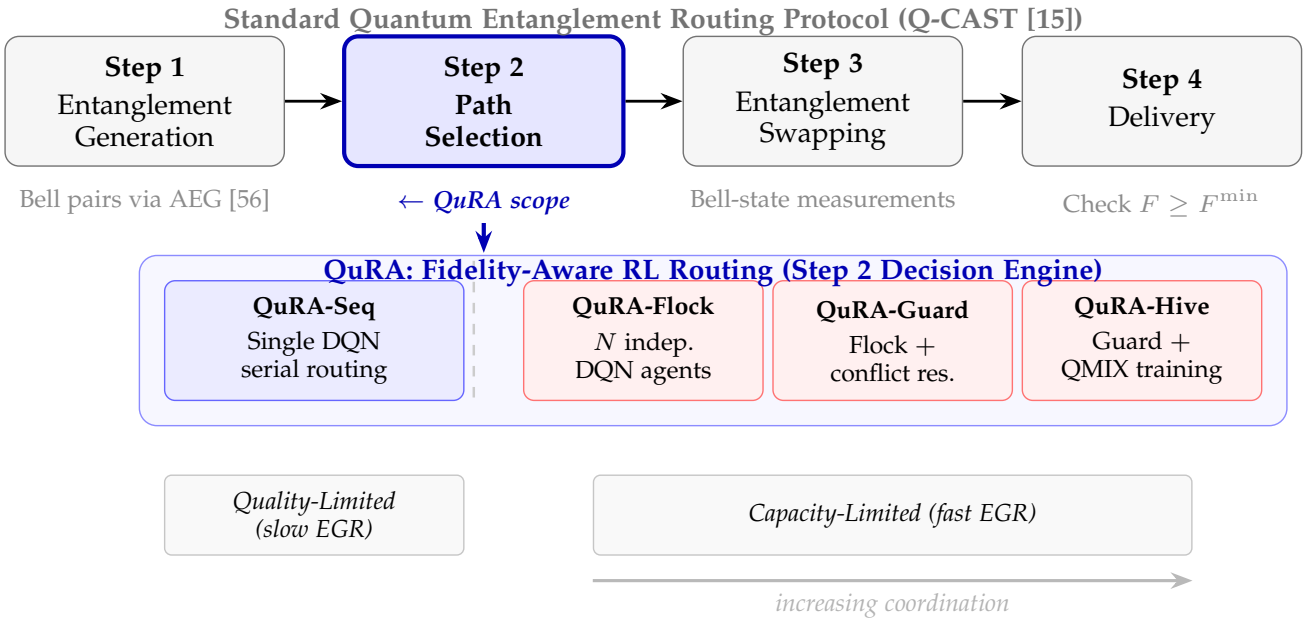


Fig. 4. QuRA within the four-step quantum entanglement routing protocol (Q-CAST [15]). Step 1 generates Bell pairs on each link via AEG [56]; Step 3 performs entanglement swapping; Step 4 delivers the end-to-end pair if $F \geq F^{\min}$. **QuRA operates at Step 2 (path selection)**, the decision-making bottleneck. Depending on the operating regime, QuRA applies one of two strategies: in the quality-limited regime (slow entanglement generation), QuRA-Seq uses a single DQN for serial, high-precision routing; in the capacity-limited regime (fast entanglement generation), three parallel variants (Flock, Guard, Hive) route all N requests simultaneously, with progressively stronger inter-agent coordination.

Action masking. At each step, a binary mask $\mathbf{m}_x$ filters all invalid actions before Q-value computation, ensuring the agent never selects an unavailable link, a previously visited node, or a link that would violate fidelity constraints. For each request r with current node c_r and visited set P_r, the valid next-hop candidates are:

$$\mathcal{N}_r = \{j \mid A_{c_r j} \geq 1,\ j \notin P_r,\ j \neq c_r\}. \tag{6}$$

Topology generalization. All variants are trained using the maximum expected number of nodes and requests, fixing input/output dimensions. At inference, smaller networks are handled by masking unused entries, enabling the same trained model to generalize across topologies without retraining, verified empirically on the 10×10 grid and the real-world Surfnet core topology in Section 5.

4.3 Sequential Variant

The sequential variant, detailed in [20], uses a single DQRL agent operating via a centralized controller. At each step m within time slot t, the agent observes the *global* network state and selects one *(request, next-hop)* pair, processing requests one at a time.

Global state representation. The agent observes a composite state $S_m^{(t)} = [R_m^{(t)}; A_m^{(t)}; D^{(t)}; Q^{(t)}]$: remaining requests $R_m^{(t)}$ (an $M \times N$ matrix encoding current and destination nodes), topology $A_m^{(t)}$ (adjacency matrix of available entangled links), edge lengths $D^{(t)}$ (physical distances), and swap probabilities $Q^{(t)}$ (per-node success rates). This *global* view gives the single agent complete information for optimizing each individual path.

Action selection. The agent selects the highest-Q valid action following the standard DQN action-selection scheme [58]:

$$a^* = \arg\max_{a \in \mathcal{A}} Q(s,a) \cdot \mathbf{m}_x(a), \tag{7}$$

decoded to a (request, next-hop) pair via:

$$r = \left\lfloor \frac{a}{N} \right\rfloor, \qquad j = a \bmod N. \tag{8}$$

Reward function. The reward combines immediate per-step feedback with a delayed episode-level signal:

$$r_m^{(t)} = \begin{cases} +10 & \text{if request reaches destination,} \\ -1 & \text{if routing is in progress,} \\ -10 & \text{if request fails (fidelity or loop),} \end{cases} \tag{9}$$

$$R_m^{(t)} = r_m^{(t)} \cdot \lambda + \alpha \cdot N_{\text{success}} + \beta \cdot \bar{F}, \tag{10}$$

where N_{success} is the number of served requests, $\bar{F}$ is average fidelity, and λ, α, β are tunable hyperparameters. The same coefficients (α, β) reappear as team weights in the parallel variants (Eq. (13)).

Architecture and training. The agent is a Double DQN [59] with two hidden layers, trained with experience replay [60] and a periodically synchronized target network. Training details and hyperparameters are reported in Section 5.

The agent architecture consists of fully connected layers that take the concatenated global state as input and output Q-values masked by the valid action set. At each step, the agent alternates between requests, selecting the next hop for one request while links available to others sit idle and decohere, the structural bottleneck motivating parallel execution.

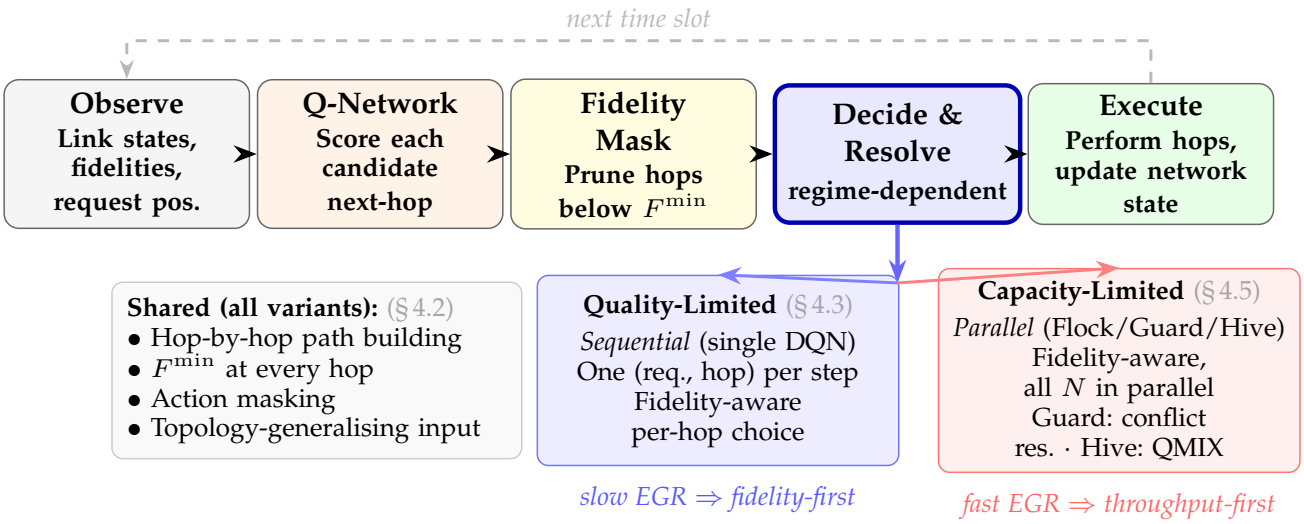


Fig. 5. QuRA decision cycle executed every time slot. Every variant shares the same five-stage pipeline (Observe → Q-Network → Fidelity Mask → Decide & Resolve → Execute) and the same set of shared routing principles (left panel). The two operating modes diverge only at the **Decide & Resolve** stage: in the quality-limited regime (slow EGR), a single sequential DQN agent picks one (request, next-hop) pair per step, maximising per-request fidelity; in the capacity-limited regime (fast EGR), N parallel agents act simultaneously: QuRA-Flock independently, QuRA-Guard with post-hoc conflict resolution, and QuRA-Hive with QMIX cooperative training.

Performance summary. We compare QuRA-Seq against two standard baselines. **Fid-ILP** is a fidelity-aware integer linear program that optimises path selection for each request subject to the $F^{\min}$ constraint; it is optimal when tractable but its runtime grows exponentially with network size. **EBSPA** (Entanglement-Based Shortest Path Algorithm) is a scalable heuristic that greedily picks the shortest path that still satisfies the fidelity threshold. QuRA-Seq matches Fid-ILP success rate within 5% while running 79% faster, and outperforms EBSPA by up to 90% at $F^{\min} = 0.7$. Runtime scales linearly with the number of requests, versus exponential for ILP. These results establish QuRA-Seq as the strongest baseline for the quality-limited regime.

4.4 Structural Bottleneck and Regime Transition

Despite strong per-request performance, sequential execution introduces a structural bottleneck under high concurrent load.

The idle-link problem. While the agent attends to request r_i, links for $r_j, r_k, \ldots$ decohere at rate $e^{-\Delta t/T_c}$ per slot (Eq. (2)). Links dropping below $F^{\min}$ are permanently lost. The expected idle-link waste grows as $O(N \cdot W)$. This is not a computational bottleneck (per-step runtime is $O(1)$ in nodes) but an *execution model* bottleneck: serialization causes permanent resource loss.

Formal characterization. Under sequential QuRA, expected link utilization per step is:

$$\mathbb{E}[L_{\text{used}}(t)] \;\leq\; \bar{d}, \tag{11}$$

where $\bar{d}$ is the average node degree. Under parallel execution, N agents infer simultaneously, scaling utilization to $N \cdot \bar{d}$.

Regime transition. The operating regime is determined by two physical parameters: the entanglement generation rate (EGR) and the available classical compute budget. When EGR is slow [21], the network produces few links per slot, idle-link waste is negligible, and sequential routing is optimal. As EGR increases through multiplexed sources [22] and industrialized platforms [23], the link supply exceeds what a single agent can consume within the coherence window. The crossover occurs when EGR $\times\, T_c > \bar{d}$ per slot; above this threshold, parallel execution becomes essential.

4.5 Parallel Multi-Agent Variants

The bottleneck motivates assigning one dedicated RL agent per routing request, enabling all N agents to infer simultaneously within each time slot. While sequential QuRA processes one request at a time (leaving links idle and subject to decoherence), the parallel framework processes all requests concurrently, achieving full link utilization.

All parallel variants share the same observation space, reward structure, and system architecture; they differ only in how agents coordinate link claims.

4.5.1 *Shared Parallel Design*

Observation space. Each parallel agent i observes:

$$o_i^{(t)} = \left[c_i^{(t)};\; d_i;\; A^{(t)};\; \mathbf{p}_{-i}^{(t)}\right], \tag{12}$$

where $c_i^{(t)}$ is the current node, d_i the destination, $A^{(t)} \in \mathbb{R}^{|V|\times|V|}$ is the full network adjacency matrix (shared across agents), and $\mathbf{p}_{-i}^{(t)} \in \mathbb{R}^{N-1}$ encodes the positions of all other agents. The observation is agent-specific through $(c_i, d_i, \mathbf{p}_{-i})$ despite sharing the global topology.

Reward structure. The reward for each parallel agent mirrors the sequential variant (Eq. (9) and Eq. (10)):

$$R_i = r_i^{(t)} \cdot \lambda + \alpha \cdot N_{\text{success}} + \beta \cdot \bar{F}. \tag{13}$$

The per-step term $r_i^{(t)} \cdot \lambda$ provides individual credit for each agent's hop decisions. The shared terms $\alpha \cdot N_{\text{success}}$ and $\beta \cdot \bar{F}$ act as a *team bonus* R_{team} that aligns all agents with the same global objective: complete as many requests as possible

while keeping mean end-to-end fidelity high. This shared signal prevents individually greedy routing from degrading aggregate performance.

Simultaneous action selection. At each time slot t, all N agents select next-hop actions concurrently. Each agent i computes Q-values for all valid next-hop candidates given its observation $o_i^{(t)}$ and selects the action with the highest masked Q-value:

$$a_i^{(t)} = \arg\max_a Q_{\theta_i}(o_i^{(t)}, a), \quad \forall\, i \in \{1, \ldots, N\}, \tag{14}$$

where Q_{θ_i} is agent i's Q-network with parameters θ_i, and the $\arg\max$ is taken over the action-masked valid next-hop set $\mathcal{N}_i$ (Eq. (6)). All N actions are computed in a single parallel batch, eliminating the sequential $O(N)$ inference bottleneck.

System architecture. The CTDE realization uses N parallel inference instances that read shared network state and execute concurrently, while a single training module updates weights asynchronously. No inter-agent messaging occurs at inference time. Fig. 5 shows the decision-cycle pipeline and the two operating modes.

4.5.2 QuRA-Flock: No Coordination

Each agent independently selects its next hop:

$$a_i^* = \arg\max_{a \in \mathcal{A}_i} Q_{\theta_i}(o_i^{(t)}, a). \tag{15}$$

When multiple agents claim the same link, the lowest-indexed agent succeeds; others retry next slot. Flock quantifies the throughput gain from parallel execution alone, providing a lower bound on the benefit of learned coordination.

4.5.3 QuRA-Guard: Conflict Resolution

Guard adds post-hoc conflict resolution. After all agents submit actions, a detector identifies contested links. For each, a priority score determines the winner:

$$\text{score}_i = \mu \cdot Q_{\theta_i}(o_i^{(t)}, a_i^*) + (1 - \mu) \cdot \frac{1}{\text{SP}_i}, \tag{16}$$

where SP_i is remaining shortest-path distance and μ balances quality against urgency. The highest-scoring agent keeps the link; others redirect to their next-best uncontested hop. Let $\mathcal{C}^{(t)}$ denote the set of contested links at slot t. The resolved action is:

$$a_i^{\text{CR}} = \begin{cases} a_i^* & \text{if uncontested or winner,} \\ \arg\max\limits_{a \in \mathcal{A}_i \setminus \mathcal{C}^{(t)}} Q_{\theta_i}(o_i, a) & \text{otherwise.} \end{cases} \tag{17}$$

If no uncontested valid hop exists, the agent waits one slot. Over-reservation followed by post-hoc resolution is deliberately aggressive: agents pursue locally optimal hops, paying resolution cost only when conflicts occur.

Fig. 6 illustrates this on a small 2×3 grid with two concurrent requests r_1 and r_2. Both agents independently pick the same next-hop link $2 \to 5$, creating a conflict (Fig. 6a). The priority scores are computed from Eq. (16) with $\mu = 0.5$: r_1 scores 0.58 and r_2 scores 0.57 (Fig. 6b), so r_1 keeps link $2 \to 5$ and r_2 is redirected to its next-best uncontested hop $3 \to 6$ (Fig. 6c). Both requests then advance in the same slot instead of one retrying.

4.5.4 QuRA-Hive: QMIX Cooperative Training

Why QMIX. QMIX [25] is a well-established value-decomposition method for cooperative MARL with centralized training and decentralized execution, and it has been shown to be a strong fit for multi-agent routing and resource-allocation tasks in classical networks [47]. The same properties apply here: agents must coordinate link claims jointly during training but act independently at inference time to keep runtime cost low. Hive builds on Guard and adds a QMIX mixing network during training. The key question: *where does centralized training help beyond reactive conflict resolution?*

Guard resolves conflicts reactively, but under high load agents enter conflict cycles: two agents alternately claiming the same link. QMIX enables *proactive coordination*: agents learn to anticipate contested links and route around them. Three benefits: (1) conflict cycle breaking, (2) load-aware path diversity, and (3) joint fidelity and throughput optimization via the global reward.

Each agent i maps its observation $o_i^{(t)}$ to a vector of Q-values over valid next-hop actions through its per-agent Q-network Q_{θ_i} with parameters θ_i:

$$\mathbf{q}_i^{(t)} = Q_{\theta_i}(o_i^{(t)}) \in \mathbb{R}^{|\mathcal{A}_i|}, \tag{18}$$

where $|\mathcal{A}_i|$ is the number of valid actions for agent i. During training, a mixing network ϕ with non-negative weights, conditioned on the full global state $S^{(t)}$ (the concatenation of all agents' observations), combines the per-agent Q-values into a joint action-value:

$$Q_{\text{tot}}\Big(\mathbf{a}^{(t)}, S^{(t)}\Big) = \phi\Big(q_1^{(t)}, \ldots, q_N^{(t)};\ S^{(t)}\Big), \tag{19}$$

where $\mathbf{a}^{(t)} = (a_1^{(t)}, \ldots, a_N^{(t)})$ is the joint action vector and $q_i^{(t)} = Q_{\theta_i}(o_i^{(t)}, a_i^{(t)})$ is agent i's selected Q-value. The mixing network enforces a monotonicity constraint:

$$\frac{\partial\, Q_{\text{tot}}}{\partial\, q_i^{(t)}} \geq 0, \quad \forall\, i \in \{1, \ldots, N\}, \tag{20}$$

which guarantees the Individual-Global-Max (IGM) property [61]: the joint greedy action $\arg\max_{\mathbf{a}} Q_{\text{tot}}$ decomposes into per-agent greedy actions $\arg\max_{a_i} q_i$, enabling decentralized execution. The mixing network and all per-agent Q-networks are trained jointly by minimizing the temporal-difference loss [42], [62]:

$$\mathcal{L}(\boldsymbol{\theta}, \phi) = \mathbb{E}\Big[\Big(R_{\text{team}} + \gamma \max_{\mathbf{a}'} Q^-_{\text{tot}}(\mathbf{a}', S^{(t+1)}) - Q_{\text{tot}}(\mathbf{a}^{(t)}, S^{(t)})\Big)^2\Big], \tag{21}$$

where $\boldsymbol{\theta} = \{\theta_1, \ldots, \theta_N\}$ are all agent network parameters, γ is the discount factor, $R_{\text{team}} = \alpha \cdot N_{\text{success}} + \beta \cdot \bar{F}$ is the team reward, and Q^-_{tot} is a periodically synchronized target network. Gradients flow through the mixer into all per-agent networks, allowing the global objective to shape individual policies. At inference the mixing network is discarded: each agent acts on local Q_{θ_i}, yielding the same runtime cost as Flock.

4.5.5 Variant Comparison

Fig. 5 shows the complete decision-cycle architecture; Fig. 4 shows where QuRA sits within the four-step Q-CAST protocol. Fig. 6 illustrates conflict resolution on a concrete example. The three variants differ only between action proposal and execution: Flock executes directly, Guard resolves conflicts post-hoc, Hive adds QMIX-trained cooperation while retaining conflict resolution.

4.6 Computational Complexity

We analyze the per-routing-window computational complexity of each variant, following the approach of DQRA [17] which shows that RL-based quantum routing has polynomial complexity in network size. Let N denote the number of concurrent requests, $\bar{L}$ the average path length in hops, $|V|$ the number of network nodes, $\bar{d}$ the average node degree, and P the number of trainable parameters in a single Q-network. For QuRA-Hive, P_ϕ denotes the additional mixing-network parameters.

Theorem 1 (Sequential Complexity)**.** *The per-window wall-clock inference complexity of sequential QuRA is* $O(N\bar{L}\bar{d}P)$.

Proof. The sequential agent routes N requests one at a time. Each request requires $\bar{L}$ hop decisions to reach its destination on average. At each hop, the agent performs one forward pass through the Q-network ($O(P)$ multiply-accumulate operations) and masks invalid actions over the current node's $\bar{d}$ neighbors ($O(\bar{d})$ comparisons). Since all $N \cdot \bar{L}$ steps are serialized, total wall-clock cost is $O(N \cdot \bar{L} \cdot (\bar{d} + P)) = O(N\bar{L}\bar{d}P)$ since $P \gg \bar{d}$ in practice. □

Theorem 2 (Parallel Complexity)**.** *The per-window wall-clock inference complexity is: (1) Flock:* $O(\bar{L}(\bar{d}P + N))$*; (2) Guard:* $O(\bar{L}(\bar{d}P + N\log N))$*; (3) Hive:* $O(\bar{L}(\bar{d}P + N))$ *(mixing network discarded).*

Proof. Each of the N parallel agents independently routes one request over $\bar{L}$ hops. At each hop, all N agents execute forward passes simultaneously: with N-way parallelism this takes $O(\bar{d}P)$ wall-clock per hop. After each hop: Flock requires $O(N)$ for result collection; Guard adds conflict detection ($O(N)$ to compare proposed actions against edge capacities) and priority sorting ($O(N \log N)$ worst case); Hive uses the same post-hoc resolution as Guard at inference (QMIX mixing network is discarded). Over $\bar{L}$ hops, the total cost follows. Training adds $O(NP + P_\phi)$ per gradient step for Hive, but this is offline. □

5 Evaluations

5.1 Evaluation Methodology

All experiments use a custom discrete-event quantum-network simulator that implements the model of Section 3 (probabilistic link generation, per-slot decoherence, Werner-state swapping, and the routing-window request model). We evaluate all methods on a 10×10 grid-based quantum network spanning a 2000×2000 km area, consistent with the evaluation setup [20] and representative of continental-scale deployments [52], [63]. We evaluate under three memory and link configurations: **(i)** 4 qubits, 3 links (resource-constrained), **(ii)** 10 qubits, 4 links (moderate), and **(iii)** 20 qubits / 6 links (resource-rich). For topology generalization we additionally evaluate on the **Surfnet core network** (20 qubits, 6 links). Entanglement swapping success probabilities are drawn from $[0.85, 0.95]$.

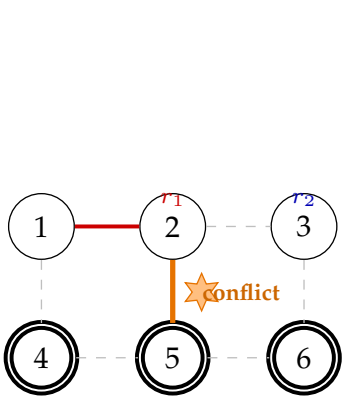
TABLE 1
Training and environment hyperparameters.

Parameter	Value
Environment	
Grid size	10×10 (100 nodes)
Area	2000×2000 km
Qubits per node	4 / 10 / 20
Links per edge	3 / 4 / 6
Swap success prob.	$U[0.85, 0.95]$
Fidelity threshold $F^{\min}$	0.7 (throughput eval.)†
Routing window W	75
Shared (all variants)	
Individual weight λ	0.3
Reward weights (α, β)	(1.0, 0.5)
Optimizer	Adam (lr $= 10^{-4}$)
Batch size	2048
Replay buffer	100,000
Sequential QuRA (QuRA-Seq)	
Q-network hidden layers	2×512
Parameters	≈2.5M
Training time	60 hours (NVIDIA T4)
Parallel Variants (Flock / Guard / Hive)	
Per-agent Q-network	2×256
Mixing network (Hive)	2×128
Conflict weight μ	0.5
Training: Flock / Guard / Hive	9 / 12 / 27 hours (NVIDIA T4)

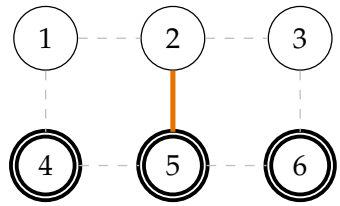
†The fidelity threshold for the SR experiment in Fig. 7 is swept from 0.5 to 0.7 to show the threshold-sensitivity analysis; all throughput experiments fix $F^{\min} = 0.7$.

We evaluate five methods across two experimental scopes. In the **sequential baseline** evaluation (Section 5.2), sequential **QuRA** [20] is compared against **Fid-ILP** (fidelity-aware ILP) and **EBSPA** (Entanglement-Based Shortest Path Algorithm) on fidelity threshold sensitivity. In the **throughput** evaluation, we compare all three parallel variants (**QuRA-Flock**, **QuRA-Guard**, and **QuRA-Hive**) against sequential QuRA and EBSPA; Fid-ILP is excluded from throughput experiments because its computation time grows prohibitively with request load (see Section 5.2). The primary metric for all parallel variants is throughput TP (Eq. (5)). The routing window is set to $W = 75$ requests based on the optimal window analysis in Section 5.5. **All throughput experiments use a fixed fidelity threshold of** $F^{\min} = 0.7$; paths delivered below this threshold are not counted toward throughput. Table 1 lists all training and environment hyperparameters.

Why ILP is excluded from throughput evaluation. Prior work [20] showed that ILP, while optimal, is not scalable: its computation time grows prohibitively with network size, making it impractical for real-time routing. Since the parallel evaluation focuses on throughput under high concurrent load ($N = 75$ requests on a 100-node network), ILP's execution time renders it non-comparable. EBSPA serves as the scalable heuristic baseline for all throughput experiments.

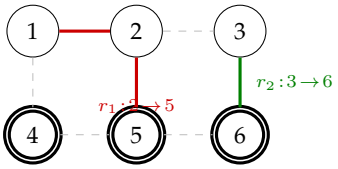


(a) Conflict detected: agents r_1 (at node 2) and r_2 (at node 3) both select link $2 \to 5$ as their next hop, creating a contested link (orange).



Agent	Q	1/SP	Score
r_1	0.82	0.33	**0.58**
r_2	0.65	0.50	0.57
r_1 wins (0.58 > 0.57)			

(b) Priority scoring resolves the conflict using $\text{score}_i = \mu Q_i + (1-\mu)/\text{SP}_i$. Agent r_1 scores 0.58 vs. r_2's 0.57 and wins.



(c) After resolution: r_1 proceeds along $2 \to 5$ (red). Agent r_2 is redirected to its next-best uncontested hop $3 \to 6$ (green).

Fig. 6. Conflict resolution example on a 2×3 grid with two concurrent requests. The priority scoring mechanism (Eq. (16)) balances Q-value quality against shortest-path urgency to determine which agent retains the contested link.

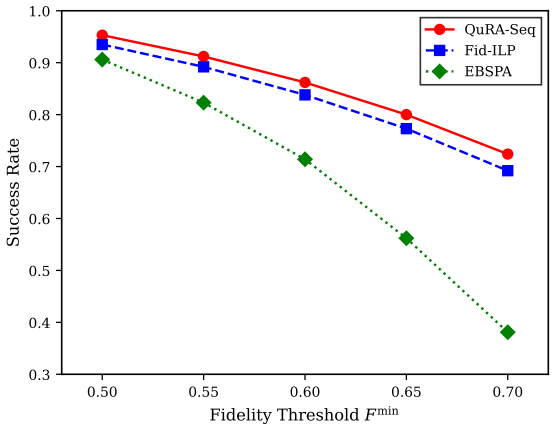


Fig. 7. Fidelity-aware routing advantage. QuRA's success rate improvement over Fid-ILP and EBSPA grows with stricter fidelity thresholds, reaching 90% over EBSPA at $F^{\min} = 0.7$. This confirms fidelity-aware path selection as the foundation of QuRA.

5.2 Sequential QuRA Baseline

Sequential QuRA [20] matches Fid-ILP success rate within 5% while running 79% faster than Fid-ILP, and outperforms EBSPA by up to 90% at $F^{\min} = 0.7$ (Fig. 7). These results confirm that sequential QuRA is well-optimized for the quality-limited regime. They also reveal the ceiling of sequential execution: beyond confirming fidelity compliance, adding more requests does not improve throughput because idle-link decoherence between sequential decisions eliminates available capacity. This ceiling motivates the parallel extension evaluated in the following sections.

5.3 Impact of Network Resources on Throughput

Figures 8(a) and 8(b) show throughput under limited resources; Fig. 9(a) and 9(b) show resource-rich configurations including topology generalization.

4 qubits, 3 links (Fig. 8(a)). Under tight constraints, all parallel variants substantially outperform sequential QuRA, which remains near zero throughout. EBSPA peaks at ≈ 47 at low load but degrades sharply under contention. QuRA-Hive maintains the highest sustained throughput among fidelity-aware methods. Sequential QuRA confirms the idle-link decoherence bottleneck: while the agent attends to one request, the remaining $N-1$ waiting requests hold links that decohere at rate $e^{-\Delta t/T_c}$ per idle slot (Eq. (2)). With a routing window of $W = 75$ concurrent requests, cumulative decoherence during sequential processing exhausts usable links before most paths complete, yielding near-zero throughput regardless of load.

10 qubits, 4 links (Fig. 8(b)). Peak throughput rises to $\approx$ 55–58 at 25 requests. QuRA-Hive sustains ≈ 48 at 150 requests vs. QuRA-Guard (≈ 40) and QuRA-Flock (≈ 30). The throughput gap between sequential QuRA and all parallel variants is consistent across the full load range, again reflecting that sequential processing cannot prevent idle-link decoherence under concurrent load.

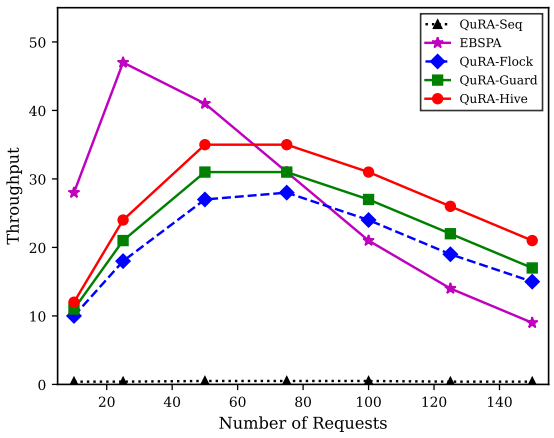
20 qubits, 6 links (Fig. 9(a)). In the resource-rich regime, parallel variants exhibit a qualitatively different behavior: throughput rises with load, peaking near 100–110 before plateauing at the network's physical link capacity. QuRA-Hive achieves the highest plateau (≈ 105) and degrades most gracefully at high load ($\approx$93 at 150 requests), confirming that QMIX coordination quality scales with available resources.

5.4 Topology Generalization: Surfnet Core Network

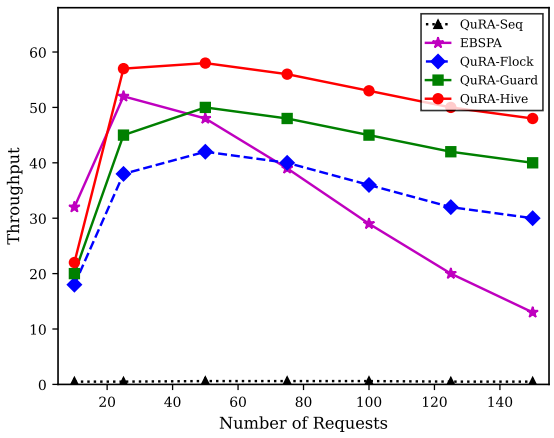
Figure 9(b) shows results on the Surfnet core network. All three parallel variants generalize without retraining. QuRA-Hive achieves peak throughput ≈ 195 at 75 requests, substantially higher than QuRA-Guard ($\approx$175) and QuRA-Flock (≈ 117). The larger QuRA-Hive vs. QuRA-Guard gap on Surfnet compared to the grid confirms that QMIX's proactive path diversity is more valuable on irregular topologies, where greedy agents converge on the same bottleneck links more frequently.

5.5 Optimal Window Size Analysis

The routing window size W controls parallelism versus contention: increasing W serves more requests simultaneously

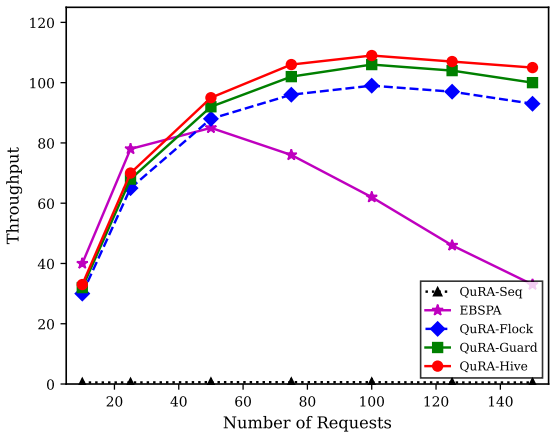


(a) 4q, 3 links

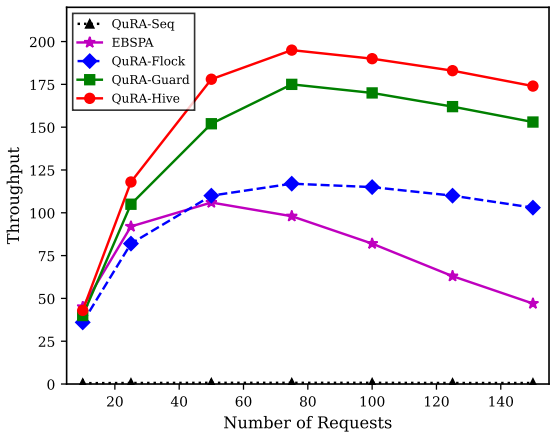


(b) 10q, 4 links

Fig. 8. Throughput vs. request load under limited resources ($F^{\min} = 0.7$). (a) 4 qubits, 3 links: all parallel variants substantially outperform sequential QuRA (near-zero throughput); EBSPA peaks early but degrades under contention; QuRA-Hive sustains the highest fidelity-aware throughput. (b) 10 qubits, 4 links: peak throughput rises to ≈55–58 and QuRA-Hive's advantage over Flock/Guard grows with load.



(a) 20q, 6 links (10×10 grid)



(b) 20q, 6 links (Surfnet)

Fig. 9. Throughput under resource-rich configurations ($F^{\min} = 0.7$). (a) On the 10×10 grid, throughput scales to ≈105 before plateauing at physical link capacity; QuRA-Hive degrades most gracefully at high load. (b) On Surfnet (irregular topology), all variants generalize without retraining; QuRA-Hive achieves ≈195 peak throughput, confirming QMIX's path diversity advantage on bottleneck-prone topologies.

but increases link competition. We derive the optimal W^* analytically and validate empirically.

Analytical derivation. Throughput as a function of W is:

$$T(W) = \frac{W_{\text{succ}}}{t_{\text{dec}}(W) + t_{\text{exec}}(W)}, \tag{22}$$

where $t_{\text{dec}} = a + bW$ is the decision time (growing linearly with concurrency management) and $t_{\text{exec}} = c + dW$ is execution time (growing with conflicts and quantum operations). The number of successfully served requests follows an exponential decay in the window size, a model we validate empirically:

$$W_{\text{succ}} = We^{-\kappa W}, \tag{23}$$

where $\kappa > 0$ is the network's congestion coefficient. We do not assume κ a priori; instead, it is measured from a calibration sweep by plotting $\log(\text{SR})$ against W and fitting the slope (Fig. 10(b)). The linear relationship $\log(W_{\text{succ}}/W) = -\kappa W$ confirms the exponential model. Maximizing:

$$\frac{d}{dW}(We^{-\kappa W}) = e^{-\kappa W}(1 - \kappa W) = 0 \tag{24}$$

gives:

$$\boxed{W^* = \frac{1}{\kappa}}. \tag{25}$$

Physical upper bound. The congestion-optimal $W^* = 1/\kappa$ is an upper bound based on the throughput model alone. In practice, W^* is further constrained by network resources. Let L_Q denote the total available quantum links and $\bar{L}$ the average path length (in hops). With $|V|$ nodes and M qubits per node:

$$W^*_{\max} = \min\left(\frac{L_Q}{\bar{L}},\ \frac{|V|M}{\bar{L}/2}\right). \tag{26}$$

The effective optimal window is therefore $W^*_{\text{eff}} = \min(1/\kappa,\ W^*_{\max})$. Both κ and $W^*_{\max}$ are network-specific:

κ depends on topology, traffic patterns, and swap success probabilities, while $W^*_{\max}$ depends on qubit and link resources. For our 10×10 grid ($L_Q \in [720, 1440]$, $\bar{L} \approx 8$):

$$W^*_{\max} \approx 62\text{–}93. \tag{27}$$

Empirical validation. Figure 10 validates the model on our specific topology. Fig. 10(b) confirms the linear relationship $\log(\text{SR}) = -\kappa W$, yielding $\kappa \approx 0.013$ and $W^* = 1/\kappa \approx 75$. Since this falls within the resource bound $W^*_{\max} \approx 62$–93, we use $W = 75$ throughout. Other network configurations would yield different κ and $W^*_{\max}$ values, requiring their own calibration.

5.6 Training Convergence

QuRA-Flock converges fastest (≈9 hours), QuRA-Guard at ≈12 hours, and QuRA-Hive at ≈27 hours, reflecting the complexity of joint QMIX training. Despite the longer training time, Hive achieves the highest final throughput. Sequential QuRA flatlines near zero throughput throughout training, confirming that the gap is architectural: no amount of training can eliminate the idle-link decoherence waste inherent to sequential request processing.

5.7 Inference Behavior: Conflicts and Runtime

Fig. 11(a) shows overbooking conflicts $C = \sum_l \max(0, \text{Demand}_l - \text{Capacity}_l)$ over training time (hours). Both variants start at ≈84 conflicts. QuRA-Guard converges to ≈36 by hour 21; QuRA-Hive achieves ≈9 by hour 24, a $4\times$ reduction, validating that QMIX's proactive coordination breaks conflict cycles that rule-based resolution cannot.

Fig. 11(b) shows algorithm runtime vs. request load. QuRA-Seq scales linearly with the number of requests. All three parallel variants (QuRA-Flock, QuRA-Guard, and QuRA-Hive) achieve near-constant runtime and are nearly indistinguishable, as the QMIX mixing network is discarded at inference leaving only per-agent forward passes. EBSPA falls between the sequential and parallel groups.

5.8 Ablation Study and Hyperparameter Sensitivity

We conduct ablation experiments to quantify the contribution of each design component and characterize sensitivity to key hyperparameters. All ablations use the 10 qubits, 4 links configuration at 75 requests per time slot unless otherwise stated.

Coordination mechanism ablation. Reading the three parallel variants at $N = 75$ in Fig. 8(b), removing all coordination (Flock) yields 40 TP. Adding conflict resolution alone (Guard) adds +8 TP (+20%). Adding QMIX training on top of conflict resolution (Hive) adds another +8 TP (+17% over Guard, +40% over Flock). This confirms that conflict resolution and QMIX training contribute roughly equally to throughput, but through different mechanisms: conflict resolution prevents immediate link waste, while QMIX produces proactive path diversity that reduces conflicts before they occur (Fig. 11(a)).

To further isolate the QMIX contribution, we evaluate a variant that uses QMIX training but removes the post-hoc conflict resolution mechanism at inference time ("Hive w/o CR"). This variant achieves 50 TP with 22 conflicts, outperforming Flock (+25%) but underperforming the full Hive (56 TP, 9 conflicts). This demonstrates that QMIX training alone substantially reduces conflicts through learned path diversity, but the residual conflicts still benefit from post-hoc resolution. The full Guard+QMIX combination is therefore not redundant: the two mechanisms are complementary.

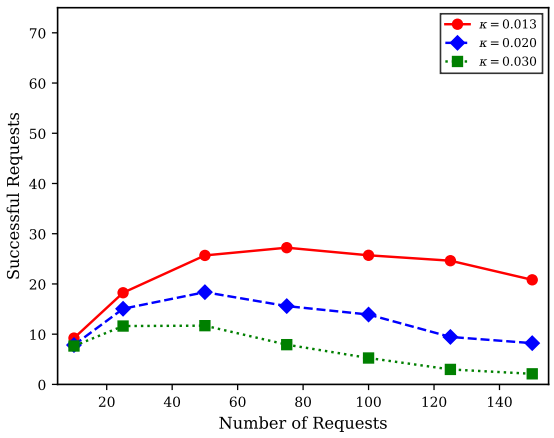
Conflict priority weight μ. The hyperparameter μ in Eq. (16) controls the balance between Q-value-based priority ($\mu = 1$) and shortest-path urgency ($\mu = 0$) during conflict resolution. We sweep $\mu \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$ for both QuRA-Guard and QuRA-Hive. QuRA-Guard is sensitive to μ: throughput ranges from 44 ($\mu = 0$) to 48 ($\mu = 0.5$) and back to 45 ($\mu = 1$), confirming that neither pure urgency nor pure Q-value priority is optimal. QuRA-Hive is markedly less sensitive: throughput ranges from 54 to 56 across all μ values, because QMIX training has already internalized coordination, making the post-hoc priority rule less consequential (Fig. 12(a)). We use $\mu = 0.5$ in all reported results.

Team reward weights (α, β). The team bonus R_{team} (Eq. (13)) balances success count (α) and fidelity (β). Setting $\alpha = 0$ (no success-count signal) degrades QuRA-Hive throughput by ≈28%, confirming that the success-count term is the primary driver of throughput; without it, agents lose their main incentive to complete requests within the window. Setting $\beta = 0$ (no fidelity signal) increases throughput by ≈5% but causes 12% of delivered paths to violate $F^{\min}$, demonstrating the fidelity regularisation role of β. The default $(\alpha, \beta) = (1.0, 0.5)$ achieves the best throughput–fidelity balance (Fig. 12(b)).

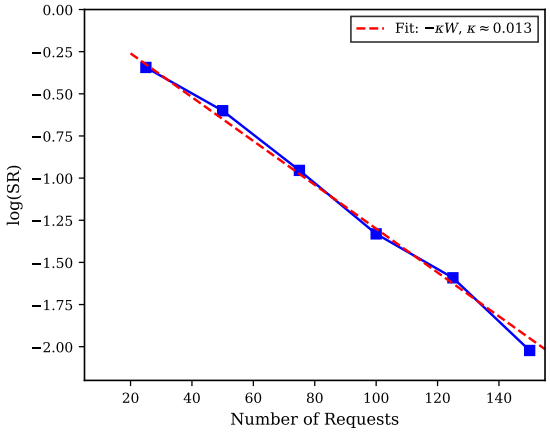
5.9 Discussion and Limitations

Comparison with existing RL baselines. RELiQ [43] and QuDQN [44] are the most closely related RL-based methods, yet direct throughput comparison is not straightforward. RELiQ assigns RL agents per network *node* rather than per request, targeting a different action decomposition; it does not report throughput under concurrent load with per-hop fidelity enforcement, making its numbers incomparable on our metrics. QuDQN focuses on a single source-destination pair with explicit purification scheduling, a richer per-request formulation but one that does not extend to the high-concurrency multi-request setting evaluated here. Adapting either system to our evaluation protocol would require substantial re-implementation with design choices not specified in the original papers; we therefore treat EBSPA as the scalable heuristic baseline and sequential QuRA as the RL baseline, consistent with prior work [20].

Centralized global state and deployment overhead. The sequential and parallel variants both assume access to the global network topology $A^{(t)}$ at each time step. In practice, assembling this state requires a classical control plane that collects link availability from all nodes via low-latency signaling. For a 10×10 network the adjacency matrix is 100×100, amounting to at most a few kilobytes per time slot, which is manageable for continental-scale networks using existing fiber-coupled control channels. The main deployment cost is latency: the control plane round-trip must complete within one coherence window T_c. For

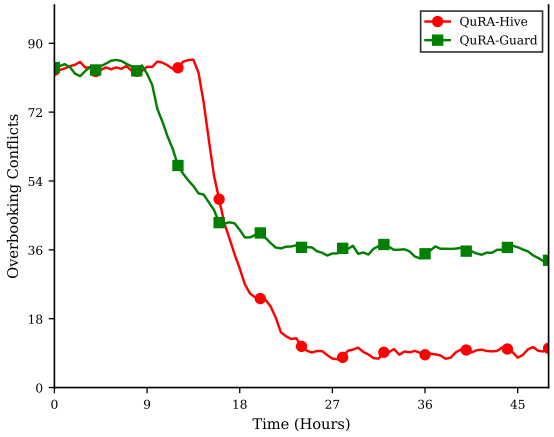


(a) Analytical model

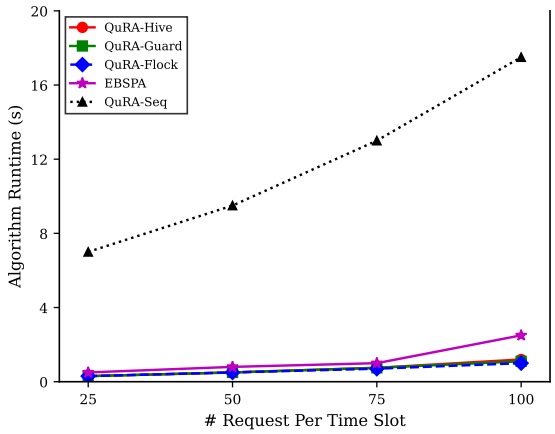


(b) Empirical validation (10×10 grid)

Fig. 10. Routing window size analysis. (a) Analytical throughput model $W_{\text{succ}} = We^{-\kappa W}$ for three congestion coefficients κ; each curve peaks at $W^* = 1/\kappa$. (b) Empirical validation: the fitted slope $\kappa \approx 0.013$ yields $W^* \approx 75$, consistent with the resource bound $W^*_{\max} \approx 62$–93. The optimum depends on both κ (measured empirically) and $W^*_{\max}$ (physical resources).



(a) Conflict reduction over training



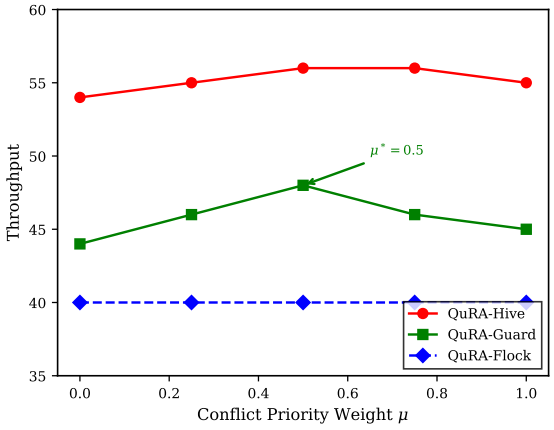
(b) Inference runtime vs. request load

Fig. 11. Inference behavior analysis. (a) QuRA-Hive reduces overbooking conflicts by $4\times$ over QuRA-Guard by hour 24, validating proactive QMIX coordination. (b) QuRA-Seq runtime scales linearly; all parallel variants (QuRA-Flock, Guard, Hive) achieve near-constant runtime as the QMIX mixer is discarded at inference.

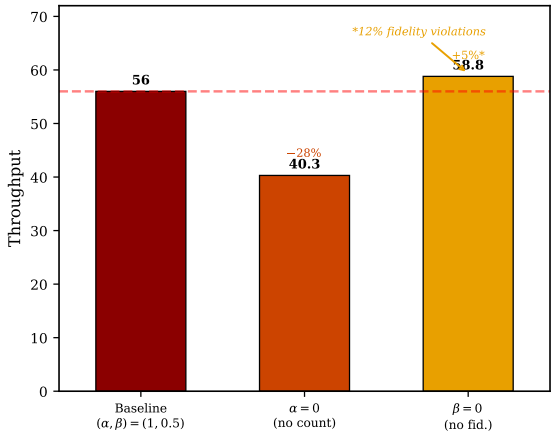
current NV-center hardware ($T_c \sim 1\,\text{s}$) this is not a bottleneck, but it may limit applicability to future hardware with sub-millisecond coherence times. Fully distributed inference without global state aggregation is a natural target for future work (Section 6).

Heterogeneous hardware and decoherence rates. All experiments use a homogeneous coherence time T_c for all nodes. Real quantum repeater hardware spans several orders of magnitude: NV-center nodes have $T_c \sim 1\,\text{s}$ [54], trapped-ion memories reach $T_c \sim 10\,\text{s}$ [50], while atomic-ensemble and photonic memories may have T_c as short as milliseconds. To assess sensitivity, we note that T_c enters our model through the decoherence penalty $e^{-\Delta t/T_c}$ (Eq. (2)): nodes with shorter T_c produce links that decohere faster, increasing pressure on the routing agent to route those requests first. Routing under heterogeneous T_c would require the agent's observation to include per-node coherence metadata and reward shaping that penalizes slow routes through low-T_c nodes. We leave a full sensitivity sweep over heterogeneous hardware profiles as future work.

Swapping success rates and current hardware. Our evaluation draws swap success probabilities from $U[0.85, 0.95]$, consistent with high-efficiency near-term photonic interfaces. Current NV-center hardware achieves swap success rates of 0.5–0.7 [54], substantially lower. At such rates, many paths fail mid-route, increasing the retry burden within the routing window and reducing effective throughput. The operating regime boundary (EGR $\times T_c > \bar{d}$) shifts accordingly: lower swap success effectively reduces the usable EGR, pushing networks toward the quality-limited regime even with fast photon generation. A regime characterization that jointly sweeps EGR, T_c, and swap success rate would more precisely map current and near-future hardware onto the sequential vs. parallel trade-off, and represents an important direction for hardware-grounded evaluation.



(a) Throughput sensitivity to conflict priority weight μ (Eq. (16)). QuRA-Guard varies from 44 to 48 TP across the sweep, peaking at $\mu = 0.5$. QuRA-Hive remains stable (54–56 TP), confirming that QMIX training internalizes coordination and reduces dependence on the post-hoc priority rule.



(b) Reward weight ablation for QuRA-Hive. Removing the success-count signal (α=0) causes a 28% drop to 40.3 TP, confirming its role as the primary throughput driver. Removing the fidelity signal (β=0) increases TP by 5% but causes 12% of delivered paths to violate $F^{\min}$.

Fig. 12. Hyperparameter sensitivity analysis (10 qubits, 4 links, 75 requests). Both analyses confirm that the default configuration ($\mu = 0.5$, $(\alpha, \beta) = (1.0, 0.5)$) achieves the best throughput–fidelity balance.

5.10 Summary

The evaluation demonstrates a clear operating regime dichotomy. Sequential QuRA achieves the highest per-request fidelity precision across all configurations, making it optimal for quality-limited networks with slow entanglement generation. However, as concurrent load increases past the crossover point (EGR $\times$ T_c $>$ $\bar{d}$), idle-link decoherence dominates: links held by waiting requests decay at rate $e^{-\Delta t/T_c}$, and the $O(N \cdot W)$ cumulative waste exhausts usable link capacity before sequential routing completes. The parallel variants eliminate this waste by processing all N requests simultaneously. QuRA-Hive achieves the highest throughput (up to $12\times$ over sequential) with the fewest residual conflicts ($4\times$ fewer than Guard), while maintaining strict fidelity compliance across all delivered paths. The optimal routing window $W^* = 1/\kappa \approx 75$ is confirmed both analytically and empirically. QMIX cooperative training and conflict resolution contribute complementary throughput gains, validated by the ablation study.

6 Conclusion

This paper presented QuRA, a family of reinforcement learning based routing algorithms for quantum networks that spans two complementary architectures matched to distinct operating regimes. The sequential single-agent variant maximizes per-request fidelity precision through centralized global-state attention, suited for quality-limited networks with slow entanglement generation. Three parallel multi-agent variants (QuRA-Flock, QuRA-Guard, QuRA-Hive) address the capacity-limited regime where idle-link decoherence dominates, with QuRA-Hive achieving the highest throughput through QMIX cooperative training.

Experiments across grid and Surfnet topologies show that QuRA-Hive delivers over $12\times$ throughput improvement over sequential QuRA while maintaining strict fidelity compliance, with $4\times$ fewer overbooking conflicts than rule-based resolution alone. The analytical optimal window $W^* = 1/\kappa$ provides calibration-based deployment guidance that generalizes to any topology.

The current design has two primary limitations that define the clearest directions for future work. First, all variants assume access to a global network state at training time and, for the sequential variant, at inference time as well. This assumption is the most significant architectural constraint: real-scale quantum networks will not support centralized state aggregation [64]. A fully distributed extension, in which each agent observes only its local neighborhood and exchanges messages only with direct neighbors during training, would remove the control-plane dependency and bridge the gap to per-node approaches like RELiQ [43], while retaining QMIX's cooperative credit assignment. Second, the current evaluation uses a fixed swap success probability range $[0.85, 0.95]$; current NV-center and trapped-ion devices achieve substantially lower rates [65]. Incorporating hardware-realistic parameters, heterogeneous decoherence rates, and purification operations will be necessary before deployment on near-term physical networks. Evaluating on larger topologies as real-world quantum network testbeds grow [66], [67] will further stress-test the generalization properties demonstrated here on Surfnet.

Acknowledgment

The work in this study was supported in part by the NSF grant 2427408.

References

[1] S. Wehner, D. Elkouss, and R. Hanson, "Quantum internet: A vision for the road ahead," *Science*, vol. 362, no. 6412, p. eaam9288, 2018.

[2] P. Jouguet, S. Kunz-Jacques, A. Leverrier, P. Grangier, and E. Diamanti, "Experimental demonstration of long-distance continuous-variable quantum key distribution," *Nature photonics*, vol. 7, no. 5, pp. 378–381, 2013.

[3] T. Islam and E. Arslan, "Quantum key distribution with single qubit transmission," in *2024 International Conference on Quantum Communications, Networking, and Computing (QCNC)*. IEEE, 2024, pp. 357–358.

[4] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, "Quantum cryptography," *Reviews of Modern Physics*, vol. 74, no. 1, pp. 145–195, 2002.

[5] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, "The security of practical quantum key distribution," *Reviews of modern physics*, vol. 81, no. 3, pp. 1301–1350, 2009.

[6] H. Buhrman and H. Röhrig, "Distributed quantum computing," in *International Symposium on Mathematical Foundations of Computer Science*. Springer, 2003, pp. 1–20.

[7] A. S. Cacciapuoti, M. Caleffi, F. Tafuri, F. S. Cataliotti, S. Gherardini, and G. Bianchi, "Quantum internet: Networking challenges in distributed quantum computing," *IEEE Network*, vol. 34, no. 1, pp. 137–143, 2019.

[8] D. Main *et al.*, "Distributed quantum computing across an optical network link," *Nature*, vol. 638, pp. 383–388, 2025.

[9] T. Chalopin, C. Bouazza, A. Evrard, V. Makhalov, D. Dreon, J. Dalibard, L. A. Sidorenkov, and S. Nascimbene, "Quantum-enhanced sensing using non-classical spin states of a highly magnetic atom," *Nature Communications*, vol. 9, no. 1, p. 4955, 2018.

[10] D. Linnemann, H. Strobel, W. Muessel, J. Schulz, R. J. Lewis-Swan, K. V. Kheruntsyan, and M. K. Oberthaler, "Quantum-enhanced sensing based on time reversal of nonlinear dynamics," *Physical Review Letters*, vol. 117, no. 1, p. 013001, 2016.

[11] H.-J. Briegel, W. Dür, J. I. Cirac, and P. Zoller, "Quantum repeaters: the role of imperfect local operations in quantum communication," *Physical Review Letters*, vol. 81, no. 26, p. 5932, 1998.

[12] W. Kozlowski, A. Dahlberg, and S. Wehner, "Designing a quantum network protocol," in *Proceedings of the 16th international conference on emerging networking experiments and technologies*, 2020, pp. 1–16.

[13] A. S. Cacciapuoti, M. Caleffi, R. Van Meter, and L. Hanzo, "When entanglement meets classical communications: Quantum teleportation for the quantum internet," *IEEE Transactions on Communications*, vol. 68, no. 6, pp. 3808–3833, 2020.

[14] R. Van Meter, T. Satoh, T. D. Ladd, W. J. Munro, and K. Nemoto, "Path selection for quantum repeater networks," *Networking Science*, vol. 3, pp. 82–95, 2013.

[15] S. Shi and C. Qian, "Concurrent entanglement routing for quantum networks: Model and designs," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 62–75.

[16] Z. Zhao, D. Bhatt, and C. Qiao, "Redundant entanglement provisioning and selection for throughput maximization in quantum networks," in *IEEE INFOCOM 2021 – IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–10.

[17] L. Le and T. N. Nguyen, "Dqra: Deep quantum routing agent for entanglement routing in quantum networks," *IEEE Transactions on Quantum Engineering*, vol. 3, pp. 1–12, 2022.

[18] P. Sharma, S. Gupta, V. Bhatia, and S. Prakash, "Deep reinforcement learning-based routing and resource assignment in quantum key distribution-secured optical networks," *IET Quantum Communication*, vol. 4, no. 3, pp. 136–145, 2023.

[19] J. Roik, K. Bartkiewicz, A. Černoch, and K. Lemr, "Routing in quantum communication networks using reinforcement machine learning," *Quantum Information Processing*, vol. 23, no. 3, p. 89, 2024.

[20] T. Islam, E. Arslan, and M. Arifuzzaman, "QuRA: Reinforcement learning based routing for quantum networks," in *2026 IEEE 23rd Consumer Communications & Networking Conference (CCNC)*. IEEE, 2026, pp. 1–6.

[21] D. P. Nadlinger, P. Drmota, B. C. Nichol, G. Araneda, D. Main, R. Srinivas, D. M. Lucas, C. J. Ballance, K. Ivanov, E. Y.-Z. Tan *et al.*, "Experimental quantum key distribution certified by Bell's theorem," *Nature*, vol. 607, pp. 682–686, 2022.

[22] A. Ruskuc, C.-J. Wu, E. Green, and A. Faraon, "Multiplexed entanglement of multi-emitter quantum network nodes," *Nature*, 2025.

[23] Nu Quantum, "Quantum networking unit (QNU): Modular platform for real-time entanglement distribution," Product announcement, 2025, mHz entanglement attempt rate, 99.7% fidelity.

[24] A. Macridin *et al.*, "Squeezed light for high-rate entanglement generation in quantum networks," *Fermilab Technical Report*, 2025, fermilab and Caltech.

[25] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 4295–4304.

[26] A. Abane, M. Cubeddu, V. S. Mai, and A. Battou, "Entanglement routing in quantum networks: a comprehensive survey," *IEEE Transactions on Quantum Engineering*, 2025.

[27] J. Illiano, M. Caleffi, A. Manzalini, and A. S. Cacciapuoti, "Quantum internet protocol stack: A comprehensive survey," *Computer Networks*, vol. 213, p. 109092, 2022.

[28] S. R. Hasan, M. Z. Chowdhury, M. Saiam, and Y. M. Jang, "Quantum communication systems: vision, protocols, applications, and challenges," *IEEE Access*, vol. 11, pp. 15 855–15 877, 2023.

[29] V. Kumar, C. Cicconetti, M. Conti, and A. Passarella, "Quantum internet: Technologies, protocols, and research challenges," *arXiv preprint arXiv:2502.01653*, 2025.

[30] M. Caleffi, "Optimal routing for quantum networks," *Ieee Access*, vol. 5, pp. 22 299–22 312, 2017.

[31] C. Li, T. Li, Y.-X. Liu, and P. Cappellaro, "Effective routing design for remote entanglement generation on quantum networks," *npj Quantum Information*, vol. 7, no. 1, p. 10, 2021.

[32] J. Li, M. Wang, K. Xue, R. Li, N. Yu, Q. Sun, and J. Lu, "Fidelity-guaranteed entanglement routing in quantum networks," *IEEE Transactions on Communications*, vol. 70, no. 10, pp. 6748–6763, 2022.

[33] H. Hu, H. Lun, Z. Deng, J. Tang, J. Li, Y. Cao, Y. Wang, Y. Liu, D. Wu, H. Yu *et al.*, "High-fidelity entanglement routing in quantum networks," *Results in Physics*, vol. 60, p. 107682, 2024.

[34] Y. Huang *et al.*, "COSP: Collaborative online scheduling protocol for quantum network routing," *International Journal of Theoretical Physics*, vol. 64, 2025.

[35] Y. Zeng, J. Zhang, J. Liu, Z. Liu, and Y. Yang, "Entanglement routing design over quantum networks," *IEEE/ACM Transactions on Networking*, vol. 32, no. 1, pp. 352–367, 2024.

[36] C. Cicconetti, M. Conti, and A. Passarella, "Request scheduling in quantum networks," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 2–17, 2021.

[37] K. Chakraborty, D. Elkouss, B. Rijsman, and S. Wehner, "Entanglement distribution in a quantum network: A multicommodity flow-based approach," *IEEE Transactions on Quantum Engineering*, vol. 1, pp. 1–21, 2020.

[38] H. Gu, Z. Li, R. Yu, X. Wang, F. Zhou, J. Liu, and G. Xue, "Fendi: Toward high-fidelity entanglement distribution in the quantum internet," *IEEE/ACM Transactions on Networking*, 2024.

[39] D. H. Nguyen, E. Hunt, D. J. Horton, T. N. Nguyen, and B.-H. Liu, "Maximizing entanglement routing rate in quantum networks: Approximation algorithms," *IEEE Transactions on Network Science and Engineering*, 2025.

[40] T. Xiao *et al.*, "Purification scheduling for entanglement throughput maximization in quantum networks," *Communications Physics*, vol. 7, pp. 1–12, 2024.

[41] Z. Yang *et al.*, "Asynchronous entanglement provisioning and routing for distributed quantum computing," in *IEEE INFOCOM 2023 – IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[42] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[43] T. Meuser *et al.*, "RELiQ: Reinforcement learning for entanglement-based routing in quantum networks," *arXiv preprint arXiv:2511.22321*, 2025.

[44] M. Jallow *et al.*, "QuDQN: Adaptive entanglement routing via deep q-networks for quantum networks," *arXiv preprint arXiv:2503.02895*, 2025.

[45] T. Chu *et al.*, "DyQNet: Optimizing dynamic entanglement routing with online request in quantum network," in *Advanced Parallel Processing Technologies (APPT 2025)*, ser. LNCS, vol. 16062. Springer, 2026.

[46] J. u. Rehman *et al.*, "Flexible entanglement routing in heterogeneous quantum networks," *TechRxiv preprint*, 2025.

[47] A. Rao *et al.*, "Multi-agent reinforcement learning for distributed resource allocation: A survey," *Artificial Intelligence Review*, 2025.

[48] S. Muralidharan, L. Li, J. Kim, N. Lütkenhaus, M. D. Lukin, and L. Jiang, "Optimal architectures for long distance quantum communication," *Scientific Reports*, vol. 6, p. 20463, 2016.

[49] K. Azuma, S. E. Economou, D. Elkouss, P. Hilaire, L. Jiang, H.-K. Lo, and I. Tzitrin, "Quantum repeaters: From quantum networks to the quantum internet," *Reviews of Modern Physics*, vol. 95, no. 4, p. 045006, 2023.

[50] N. Sangouard, C. Simon, J. Minář, H. Zbinden, H. De Riedmatten, and N. Gisin, "Long-distance entanglement distribution with single-photon sources," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 76, no. 5, p. 050301, 2007.

[51] Z. Liu, S. Marano, and M. Z. Winx, "Establishing high-fidelity entanglement in quantum repeater chains," *IEEE Journal on Selected Areas in Communications*, 2024.

[52] C. M. Knaut *et al.*, "Entanglement of nanophotonic quantum memory nodes in a telecom network," *Nature*, vol. 629, pp. 573–578, 2024.

[53] W. Dür, H.-J. Briegel, J. I. Cirac, and P. Zoller, "Quantum repeaters based on entanglement purification," *Physical Review A*, vol. 59, no. 1, p. 169, 1999.

[54] N. Kalb, A. A. Reiserer, P. C. Humphreys, J. J. Bakermans, S. J. Kamerling, N. H. Nickerson, S. C. Benjamin, D. J. Twitchen, M. Markham, and R. Hanson, "Entanglement distillation between solid-state quantum network nodes," *Science*, vol. 356, no. 6341, pp. 928–932, 2017.

[55] A. Dahlberg, M. Skrzypczyk, T. Coopmans, L. Wubben, F. Rozpędek, M. Pompili, A. Stolk, P. Pawelczak, R. Knegjens, J. de Oliveira Filho *et al.*, "A link layer protocol for quantum networks," in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2019, pp. 159–173.

[56] T. Islam, M. Arifuzzaman, and E. Arslan, "Adaptive Entanglement Generation for Quantum Routing," *arXiv preprint arXiv:2505.08958*, 2025.

[57] Y. Lee, W. Dai, D. Towsley, and D. Englund, "Quantum network utility: A framework for benchmarking quantum networks," *Proceedings of the National Academy of Sciences*, vol. 121, no. 17, p. e2314103121, 2024.

[58] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[59] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30. AAAI Press, 2016.

[60] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, no. 3–4, pp. 293–321, 1992.

[61] K. Son, D. Kim, Y. Kim, D. E. Hostallero, and Y. Yi, "QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR, 2019, pp. 5887–5896.

[62] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[63] W. Wang, K. Zhang, and J. Jing, "Large-scale quantum network over 66 orbital angular momentum optical modes," *Physical Review Letters*, vol. 125, no. 14, p. 140501, 2020.

[64] E. E. Dobbs, N. Delfosse, and A. Brodutch, "Advantage in distributed quantum computing with slow interconnects," *arXiv preprint arXiv:2512.10693*, 2025.

[65] C. Zhu, C. Zhu, and X. Wang, "Estimate distillable entanglement and quantum capacity by squeezing useless entanglement," *IEEE Journal on Selected Areas in Communications*, 2024.

[66] J. K. Søndergaard, R. B. Christensen, and P. Popovski, "Satellite-aided entanglement distribution for optimized quantum networks," *arXiv preprint arXiv:2502.18107*, 2025.

[67] D. Gao, D. Fan, C. Zha, J. Bei, G. Cai, J. Cai, S. Cao, F. Chen, J. Chen, K. Chen *et al.*, "Establishing a new benchmark in quantum computational advantage with 105-qubit zuchongzhi 3.0 processor," *Physical Review Letters*, vol. 134, no. 9, p. 090601, 2025.